Grid vs. Graph Representations for Self-Play Learning in Hive

A pre-registered comparison under limited compute

Mohamed Bechir Kefi

Independent research report (not peer-reviewed); Version v2.1-draft, October 2026

Abstract

Hive is a boardless hexagonal strategy game whose moves are (piece, destination) pairs over an ever-changing set of cells, which makes it, in principle, a natural candidate for graph neural encodings in place of the convolutional grid encodings standard in AlphaZero-style systems. We test that intuition under a pre-registered protocol frozen before any comparison run: one grid CNN and one capacity-matched (+1.5%) relational message-passing network share a rules-validated engine, one action decoder, identical training settings, a frozen three-opponent population and 250 frozen openings, and are evaluated under two budget readings (equal training examples; equal wall-clock at a pre-registered cutoff) with five independent seeds per arm and truncation reported as its own outcome. The hypothesis is rejected: the graph arm scores lower against two of the three opponents under both readings, at twice the wall-clock cost, with its deficit concentrated in converting won positions; the largest upper bound of any interval on a graph advantage is +0.028. Ablations show that the graph arm's typed edge relations are necessary for optimization stability, while its global pooling is dispensable. A second, separately pre-registered study under a warm-started loop with a three-generation replay window (five new seeds per arm) detected no graph advantage under either reading: every graph–grid interval includes zero (largest upper bounds +0.090 and +0.055), the deficit is no longer visible at equal examples and reduced at equal wall-clock, and the conversion pathology persists in three graph seeds of five at 2.34 times the cost. Both results are pre-registered and reproducible from the released records.

1 Introduction

Since AlphaZero, learning to play a board game without human examples has followed a stable recipe: a network estimates a policy and a value for each position, a tree search guided by the network produces stronger decisions than the network alone, and the games the search plays against itself become the next network's training data (Silver et al., 2018). One ingredient of the recipe is not general: how a position is presented to the network. For chess, shogi and Go the board is a fixed array, a position becomes a stack

of image-like planes, and a convolutional network reads them; the grid encoding works because every square exists at every moment, at a fixed place.

Hive has no such property. It is played with hexagonal tiles and no board: the tiles form the playing surface, the surface changes shape every turn, tiles climb on top of one another, and a move is best described as "this piece goes to that cell" over a set of cells that exists only relative to the current hive. The natural object for such a position is a graph, with cells as nodes, adjacency as edges and stacks as node attributes, and the natural network is a graph neural network, which needs no frame. Grid encodings can still be applied by unfolding the hive into a large fixed frame, at the price of anchoring choices, empty space and a very large discrete move space whose structure the network must learn from scratch.

It is tempting to conclude that a graph encoding must learn Hive better. We designed this study because the intuition cuts both ways. A graph network matches the game's native structure and carries no anchoring artifacts. But Hive's outcomes hinge on short-range surround tactics (a queen bee loses when its six neighbouring cells are occupied), the regime in which convolutional locality is strongest. Moreover, a graph over the occupied cells alone cannot express the legal destinations of a move, which are properties of empty space; the graph must carry empty candidate cells, and it is not known whether a simple message-passing network learns the sliding, climbing and connectivity constraints from such a graph better than a convolutional network learns them from the unfolded frame.

The prior evidence does not settle the question. The two direct grid-versus-graph comparisons on board games point in opposite directions: in Hex, under a value-based learner, a graph network dominated on long-range dependencies while the convolutional network stayed sharper at local patterns (Keller et al., 2023), and in chess, under self-play, a graph-attention network out-learned convolutional baselines, from a single training run per model (Rigaux and Kashima, 2024). The only AlphaZero-style study of Hive compared five board encodings, all of them grids (de Goede et al., 2022), and none of these works ran both arms under one self-play pipeline, at matched capacity, under a matched budget, with several independent runs per arm, on Hive.

Our main question (RQ-H1) is whether, at comparable training budget, a graph architecture learns a better policy than a grid architecture for base-game Hive within an AlphaZero-style self-play pipeline. The hypothesis under test, H1, states that a simple message-passing graph network, receiving the hive as a graph, reaches a higher mean score against a fixed opponent population than a grid convolutional network receiving a 32×32 unfolded frame, at equal training budget. Two secondary questions ask whether the answer changes when the budget is read as training examples or as wall-clock (RQ-H2), and which components of the graph architecture carry its behaviour (RQ-H3). The protocol, frozen before any comparison run, rejects H1 if, under both budget readings, the graph arm shows no seed-consistent advantage and the seed-level interval on the graph–grid score difference excludes a meaningful graph advantage; it states that a negative or null result is a publishable outcome.

We claim four contributions.

- The first controlled, budget-matched, multi-seed grid-versus-graph comparison for Hive, with both arms under the same AlphaZero-style pipeline, read under two budget equalisations, and a pre-registered negative answer.

- A reproducible comparison harness for frameless, stacking games: a rules engine validated independently of learning, two encoders pinned byte-exactly across two implementation languages, a shared decoder over a variable action set, truncation-aware evaluation and a frozen-artifact discipline.
- A component attribution for the graph arm, in which the typed relations determine trainability and the pooling has a null effect, with an honestly labelled two-component supplement.
- A second pre-registered comparison under the standard warm-started regime, with its protocol, equal-time cutoff and seeds fixed before any run, whose null result bounds the first study's negative one and is reported beside it without pooling.

The accompanying technical report contains the complete architectures, the raw per-seed tables, the hand-annotated corpora and the provenance of every number.

2 Background and related work

Hive. Hive (Yianni, 2010) is a two-player, perfect-information, zero-sum game played without a board. Each player owns eleven tiles in the base game: one queen bee, two spiders, two beetles, three grasshoppers and three soldier ants. Tiles are placed edge to edge, a tile entering play may not touch an opponent's tile, and the queen must enter by each player's fourth turn. Each bug type moves differently (sliding, climbing or jumping), the hive must stay connected at all times, and a sliding tile must physically pass between its neighbours. A player loses when their queen is surrounded on all six sides. Four properties make Hive awkward for methods developed on fixed boards: the geometry is frameless, tiles stack, the branching factor stabilises around 60 (roughly twice that of chess) with games that run for hundreds of plies, and legality is non-local, since whether a move exists depends on articulation points of the hive and on empty space (Kampert et al., 2021).

AlphaZero-style self-play. A single network maps a state to a policy over actions and a value for the player to move; the network guides a PUCT tree search, and the search generates the games the network is then trained on, with the root visit distribution as policy target and the game outcome as value target (Silver et al., 2018). Of the economies Wu (2020) introduced in KataGo we adopt one, playout-cap randomization, because it is representation-agnostic: most decisions are searched cheaply and not recorded, and a minority receive a full search and become training examples. Jones (2021) measured that training compute and search compute trade against each other, so an evaluation must fix the search budget identically across arms. Agarwal et al. (2021) showed that deep reinforcement learning results from a handful of training runs are routinely over-read, that comparisons with 3–5 runs are often not supported by their own data, and that uncertainty is badly underestimated below about ten runs; they prescribe interval estimates from a bootstrap over runs, which we adopt with the seed as the unit.

Grid planes versus cell graphs. The canonical encoding of the AlphaZero family is a stack of planes over the board array read by a residual convolutional network, which for Hive must be manufactured by anchoring the hive in a frame. The alternative hands the network the adjacency structure itself: in the inductive message-passing family of Hamilton et al. (2017), per-layer weights shared across all nodes embed any graph,

including graphs never seen in training, and every Hive position is an unseen graph. Plain neighbourhood aggregation ignores edge semantics, whereas in Hive the direction of a neighbour matters (a grasshopper jumps along a line; a gate is formed by the two cells flanking one direction), so our graph arm uses direction-typed relations. A graph encoding grants no invariance for free: message passing with typed edges is not rotation-invariant, receptive fields are limited by depth, and no rule of Hive is known to the network. Pointer networks (Vinyals et al., 2015) justify scoring a variable candidate set directly, the mechanism behind our shared action decoder.

Direct precedents and the evidence standard. Three works compare a grid encoder with a graph encoder on a board game; Table 1 places them, with the only AlphaZero-style Hive study, beside ours. Keller et al. (2023) ran a parameter-matched comparison on Hex (≈487K against ≈481K parameters, ≈110 A100-hours per model): the graph network made 1 error on a long-range suite where the CNNs made 31 and 36, while the CNN stayed sharper at local patterns; but the comparison ran under RainbowDQN, the CNN arm never trained under tree search, and the graph is a Hex-specific reduction in which every node is an empty cell. Rigaux and Kashima (2024) report the opposite sign for chess under self-play, with every model trained once, "comparable parameters" spanning 1.0M against 2.2M, and an action parameterisation that changes with the representation. AZ-Hive (de Goede et al., 2022) crossed five dense hex-lattice encodings with two action encodings on a 26×26 frame, each trained for 4 h with 5 repeats, found that the encoding measurably changes early learning, and had no graph representation in its design space. Together these works make a graph advantage plausible while showing that a CNN can win on local patterns, that the published positive result rests on one run per model with a confounded decoder, and that what the edges encode may matter more than graph structure itself. Hive's play is dominated by local adjacency tactics around the queen, where the Hex evidence favours the grid.

Table 1: The three direct grid-versus-graph precedents and the only prior AlphaZero-style Hive study, beside the present design. "Runs per model" is the number of independent training runs behind each reported comparison.

Work	Game	Representations compared	Runs per model	Limit for the Hive question
Keller et al. (2023)	Hex	15-layer SAGEConv (≈487K) vs ResNet and U-Net (≈481K)	1	Comparison under RainbowDQN; CNN never under self-play search; Hex-specific graph with per-node actions
Rigaux and Kashima (2024)	Chess	Edge-featured graph attention (1.0M) vs CNN planes (2.2M)	1	No seed variance; loose capacity match; decoder changes with representation
Ben-Assayag and El-Yaniv (2021)	Othello, Gomoku, Go	3-layer GIN over the square lattice vs AlphaZero CNN	5	Transfer claim under asymmetric budgets; node-only actions
AZ-Hive (de Goede et al., 2022)	Hive, base game	5 grid encodings × 2 action encodings, all CNN, 26×26 frame	5	No graph arm; conclusions on early learning speed
This study	Hive, base game	Residual CNN on a 32×32 frame (1.44 M) vs direction-typed message passing on the cell graph (1.47 M); shared decoder	5	One variant, one capacity, one machine, small budget

3 System

An engine validated independently of learning. Every arm, opponent and evaluation shares one rules engine, written in Rust in July 2026 as an engine project that predates the study. Because a rules defect would teach both arms the same wrong game and silently invalidate the comparison, we re-established its validation on the study machine on 9 September 2026, before any baseline or training work, along five lines. Perft counts match the published tables of the protocol's reference engine (Mzinga) to depth 6 for all eight game types of the Universal Hive Protocol, the depth-6 run completing in 8.32 s. The conformance harness of a second reference engine (nokamute) passed 21/21, and differential fuzzing asserted set equality of the legal-move sets with both reference engines after every ply over 27,829 positions. A corpus of 30 critical positions annotated by hand from the publisher's rules (Yianni, 2010; World Hive Tournaments rules FAQ) and committed before any engine run returned 29 passes and one setup error on the first run, resolved against the corpus, and 30/30 on the second; a five-position tactical suite was solved 5/5 by the network-free search at 400, 1,600 and 6,400 simulations. Seeded random games across all eight game types applied and undid 10.9 million transitions without an invariant violation. Finally, each encoder exists twice, in Rust inside the search and in Python inside the training loop, and the two are pinned byte-exactly by golden crosschecks (240 positions for the grid encoder, 160 of 160 for the graph encoder). None of this proves full-game perfection; it bounds five different failure modes.

The shared pipeline. Both arms are trained and evaluated by one AlphaZero-style loop with playout-cap randomization. A run comprises 10 generations of 500 self-play games on one machine with four worker threads. Self-play searches 128 simulations on a random quarter of decisions and 32 on the rest, and only full-budget decisions become records; Dirichlet noise with $\varepsilon = 0.25$ is mixed into the root prior, moves are sampled from the visit distribution for the first 12 plies, a side resigns below −0.92 (resignation is disabled in 10% of games as an audit), and a game reaching 300 plies is truncated, a fourth outcome carried through the record format, the loss (truncated records contribute no value target), the arena and every table. Training is identical for both arms: stochastic gradient descent at learning rate 0.02 under a cosine schedule, momentum 0.9, weight decay 10^{-4}, batch 256, two epochs per generation, and a loss that adds the policy cross-entropy over exactly the legal set to 0.6 times the three-way win/draw/loss cross-entropy over non-truncated records. One property of the loop must be stated plainly: the network of generation g is a fresh seeded initialisation trained on the records of generation g alone, which were produced by the search guided by the network of generation $g-1$; the driver passes neither a warm-start checkpoint nor a replay window to the trainer. The loop still improves across generations, because the search is stronger than the raw network, but no weights are carried forward and no record is reused. Seven automated checks of the data path and the training loop (from legal-set normalisation through a real forward pass to evaluation never feeding training) ran against real self-play shards before any training output was trusted, and a one-generation pilot validated the loop end to end on 10 September 2026. Neither arm received any hyperparameter search.

Two representations, one decoder. Figure 1 shows one position under both encodings and Figure 2 the two networks. The grid arm unwraps the engine's board by breadth-first traversal, centres the occupied bounding box in a 32×32 frame, and writes 77 planes: piece planes indexed jointly by owner, bug type and stack level, planes for pinned tiles,

the last-moved tile and both sides' placement regions, and constant planes for the side to move, queen liberties, ply and reserves. HiveNet, the grid network, is a residual convolutional network of 96 channels and 8 residual blocks with a global-pooling bias in blocks 2 and 5, a policy head of 28,673 logits (28 piece-slot planes over the frame plus pass) and a three-way value head; it has 1.44 M parameters. The graph arm is coordinate-free. Its nodes are the cells of the candidate set, every occupied cell and every empty cell adjacent to the hive, so that every legal destination is a node; each node carries 56 features describing its stack level by level, and a 23-feature global vector is broadcast to every node. Edges are the directed adjacencies between candidate cells, typed by the six hexagonal directions. HiveGraphNet maps node and global features to 152 channels and applies 8 relational message-passing layers, each updating a node from itself and its six directional neighbours through six direction-typed weight matrices, with a global-pooling bias (masked mean and maximum of the node embeddings) every third layer; the policy head scores each legal (piece, destination) pair from the destination node's embedding, the moving piece's source embedding and a slot embedding; the value head reads a masked mean-and-max pooling. It has 1.47 M parameters, +1.5% relative to the grid network. Both arms feed one action decoder: logits exist only for the legal (piece, destination) pairs the engine generates, the softmax runs over exactly that set, the policy target is the same root visit distribution, and tie-breaking is identical. Neither arm trains with symmetry augmentation.

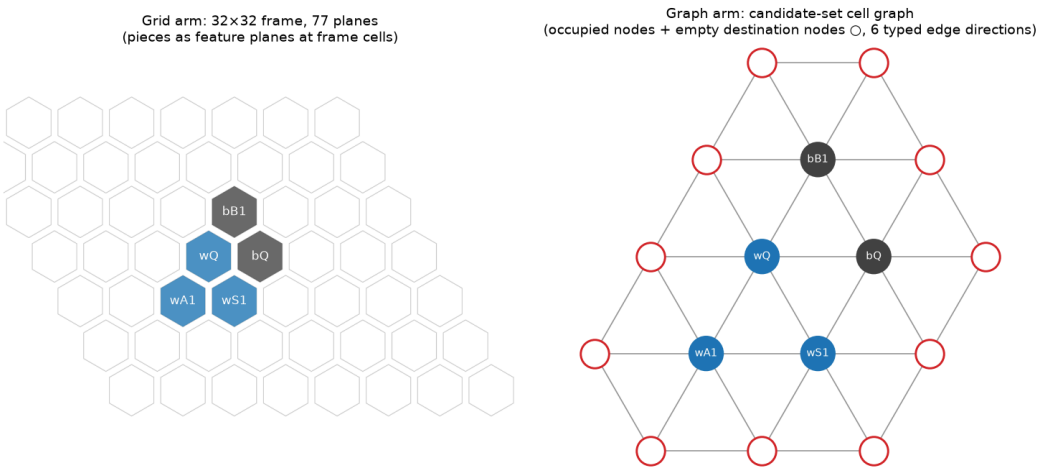


Figure 1: One position under the two encodings. Left, the grid arm's view: the position embedded in a fixed 32×32 frame of hexagonal cells, each frame cell carrying 77 feature values. Right, the graph arm's view: the cell graph over the candidate set, in which occupied cells (filled) and empty cells adjacent to the hive (open circles) are joined by edges typed with the six hexagonal directions. Both views derive from the same engine state without loss.

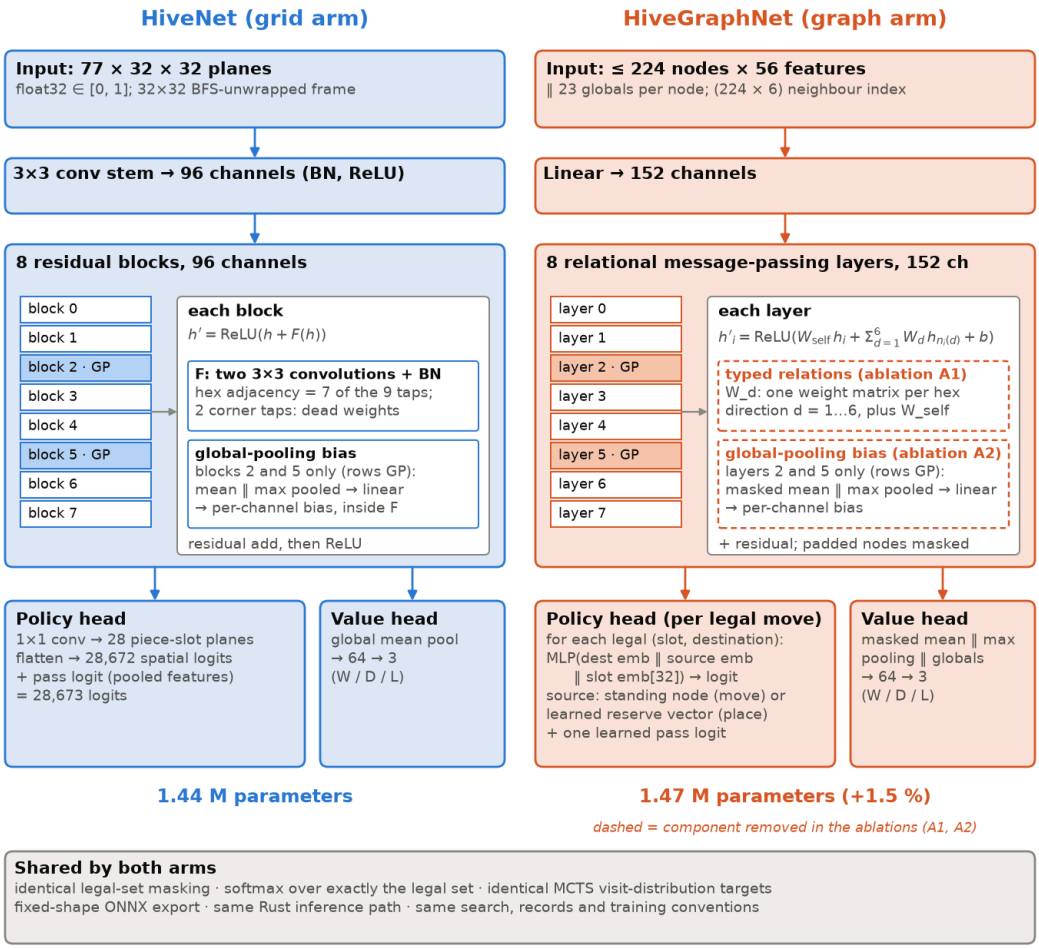


Figure 2: The two networks. Left, HiveNet (grid arm): 3×3 convolutional stem, 8 residual blocks of 96 channels with a global-pooling bias in blocks 2 and 5, a flat 28,673-way policy head and a 3-way value head, 1.44 M parameters. Right, HiveGraphNet (graph arm): input layer to 152 channels, 8 relational message-passing layers with six direction-typed matrices and a global-pooling bias every third layer, a per-candidate policy scorer and a masked mean-and-max value head, 1.47 M parameters. Both feed the shared action decoder.

Measured cost asymmetry. What could not be matched is the cost of running each network on the study machine (Apple M1 Pro, 10 cores, 16 GB). At batch size 1 the convolutional network runs fastest on the CoreML accelerator, at 2.62 ms per evaluation; the gather-heavy graph network is split by CoreML into 15 sub-graphs with 147 of 287 operators supported and runs fastest on the CPU provider, at 3.67 ms (9.84 ms on the accelerator), a ratio of ≈1.4× against the graph arm. Training throughput at batch 128 is 274 positions per second for the grid network and 138 for the graph network. We report and charge this asymmetry rather than equalise it away, since throttling one arm would manufacture a parity that no user of either representation would experience; the two budget readings account for it.

4 Experimental protocol

Frozen artifacts. A comparison on one machine with a handful of seeds leaves the experimenter great freedom after the fact, and we removed these degrees of freedom by making every commitment before the observation it could bias, each artifact frozen under a content hash and approved by the author. The opponent population and the evaluation search settings were frozen on 9 September 2026; the protocol (version 1.0, including the rejection rule), the 250 shared openings and the comparison matrix with its cutoff rule on 10 September 2026, after the pilot had re-measured the budgets with the study's own networks so that the frozen text carries no placeholders; the first comparison run started on 10 September 2026 at 12:42. The protocol declares any later change a new study, and none was made.

Design. Two arms × five seeds × 10 generations × 500 games, at 128/32 simulations per decision with playout-cap randomization; the runs are identical in everything except the state encoder and the network body. We read one campaign in two ways. Under the same-examples reading, both arms are compared at their final checkpoints, having trained on the same number of games at the same simulation budget. Under the same-wall-clock reading, each run is read at its last checkpoint completed at or before an equal-time cutoff defined by a score-free rule in the pre-registered matrix: the median full-run wall-clock of the three original grid runs, which gives the grid arm its full budget by construction and charges the graph arm its true cost. The rule was executed on 16 September 2026, from grid clocks alone and before any cross-arm number existed: median(18.77, 16.73, 19.07) = 18.77 h, evaluation time excluded for every run alike. The cutoff selected generation indices 9, 9, 8, 9, 9 for the grid runs and 3, 4, 4, 2, 5 for the graph runs (counting from 0): at equal time the graph arm had completed 3–6 of its ten generations.

Opponents, openings and evaluation. Hive has no public ladder of reference networks and no perfect-play anchor, so we evaluate against a fixed population of three opponents, all backends of the same engine: B-RND, uniform over legal moves; B-HEU, a greedy one-ply argmax of the engine's documented hand-crafted evaluation, with hash-pinned weights; and B-MCTS, the same tree search as the learned agents with uniform priors and the hand-crafted evaluation as leaf value, at 6,400 simulations per decision. A round-robin characterisation on 9 September 2026 (100 paired games per pairing) placed the heuristic at 100/0/0 and the search at 99/1/0 against legal-random, and the search at 37.5% against the heuristic; the last ordering was unexpected and was deliberately not retuned, since the freeze forbids adjusting an opponent in view of results. Games start from 250 unique legal four-ply openings generated blind by a seeded random walk and frozen under their content hash; pair i of every match plays line i once with each colour, so every arm, seed, checkpoint and opponent faces an identical opening-and-colour schedule. Final and cutoff evaluations play 100 paired games per opponent at 400 simulations per decision, with no root noise and a deterministic best move; intermediate evaluations at generations 5 and 8 play 20 games per opponent and serve only the trajectory figure. A precision check on 16 September 2026 found the seed-to-seed spread against the heuristic (SD ≈ 0.056) about twice the game noise at 100 games (SE ≈ 0.03), so the only lever was more seeds.

Statistics. The primary metric is the mean score (win 1, draw 0.5, loss 0) over the non-truncated games of a (seed, opponent) cell. The seed, one independent training run, is

the unit of analysis: games played by one trained network share its weights and are not independent samples of the method. Seed-level means and 95% intervals are percentile bootstraps with 10,000 resamples over seeds; for the graph–grid contrast the two seed sets are resampled independently, since training seeds are not paired across arms, which is conservative for a design in which both arms play the same openings. Every table shows every seed. Differences are written as graph minus grid, so a negative number favours the grid arm.

Truncation. A game reaching the 300-ply cap is never a draw. The primary score excludes truncated games; a sensitivity column scores them 0.5; two bounding treatments score every truncated game of the arm under test as a loss and as a win; and the direction of the contrast is reported under all four treatments. The "win" treatment bounds what any larger cap could do, which is why bounds replace a re-run at a larger cap. The cap sits beyond the longest search-guided game observed during profiling (202 plies).

Two-stage seed collection. The matrix pre-registered three seeds per arm with a five-seed option "if budget allows". After the three-seed analysis of 19 September 2026, the author approved on 26 September 2026 seeds 4 and 5 for both arms under three pre-commitments written before any new run: all five seeds per arm enter the final analysis regardless of the new seeds' direction; the equal-time cutoff stays at its already-computed value; and the two-stage collection is disclosed. Seeds added symmetrically under an unchanged rule are a decision about statistical power rather than a tuning channel. A run may be excluded only for a process failure, never for a bad score; no run failed, was restarted or was excluded in any campaign.

Ablations. The ablation rules were fixed before the ablation runs: at most two ablations, each removing exactly one component of the graph arm at full parity (seeds 1–3, the same budgets, frozen opponents, openings and evaluation); a capacity change inherent to the removed component is reported, never compensated; and where more than one component differs, no effect is attributed to the graph. A1 replaces the six direction-typed matrices by one shared matrix (naive adjacency, 0.54 M parameters). A2 removes the global-pooling bias from every layer (1.37 M); it was substituted on 19 September 2026 for the planned augmentation-removal ablation, which had nothing to remove, and this choice of question, made after the main result, is disclosed. Both were approved on 20 September 2026. When A1 proved untrainable, a supplement A1′ was approved on 26 September 2026: untyped edges plus a global gradient-norm clip of 1.0, the only optimizer change in the study, labelled two-component wherever it appears.

Second study. After the first study had been analysed and written up, a second protocol (version 2.0) was frozen on 13 October 2026, before any new run, to test the first objection to the verdict: that training every generation's network from a fresh initialisation on that generation's games alone had kept both arms in too early a regime. It changes exactly two things, identically in both arms: the network of generation g is initialised from generation $g-1$'s final checkpoint (weights only; optimizer and schedule restarted), and generation g trains on the records of the last three generations (500, 1,000, then 1,500 games). Everything else is the first study's, including the untuned hyperparameters (an identical small tuning budget per arm was considered and rejected at the freeze), the frozen opponents, openings and evaluation, the statistics and the rejection rule; the new runs use seeds 11 to 15; the equal-time cutoff is the median of the second study's five grid clocks, computed once before any score was read, 17.63 h from totals of 17.46, 17.95, 17.63, 17.46 and 18.52 h; and the two studies are compared descriptively, never pooled. A pilot

of two generations per arm measured the regime's cost before the freeze; the ten runs executed on 13–26 October 2026 without incident, in 296.9 h of training wall-clock, at a cost ratio of 2.34 against the grid arm.

5 Results

Verdict. Under both budget readings, the graph arm shows no seed-consistent advantage against any opponent, and every interval on the graph–grid score difference excludes a meaningful graph advantage: the largest upper bound across the six contrasts is +0.028. The pre-registered rejection rule fires and H1 is rejected. Table 2 gives the seed-level means under both readings, Table 3 the contrasts, and Figure 3 the score of every run against its training wall-clock.

Table 2: Seed-level mean score per arm and opponent with the 95% percentile bootstrap interval over the five seeds (10,000 resamples), under the same-examples reading (final checkpoints) and the same-wall-clock reading (last checkpoint within the 18.77 h cutoff). Each seed's score is the mean over decided games (win 1, draw 0.5, loss 0) of 100 paired colour-swapped games per opponent on the 250 frozen openings at 400 simulations per move; games truncated at the 300-ply cap are excluded.

Reading	Arm	vs B-RND	vs B-HEU	vs B-MCTS
Same examples	grid	0.981 [0.964, 0.994]	0.129 [0.098, 0.165]	0.124 [0.087, 0.168]
Same examples	graph	0.812 [0.710, 0.920]	0.065 [0.034, 0.100]	0.115 [0.107, 0.121]
Same wall-clock	grid	0.971 [0.950, 0.991]	0.120 [0.098, 0.142]	0.125 [0.090, 0.168]
Same wall-clock	graph	0.810 [0.697, 0.922]	0.061 [0.047, 0.081]	0.107 [0.087, 0.130]

Table 3: The pre-registered contrast: difference of seed-level mean scores (graph arm minus grid arm) with the 95% bootstrap interval over seeds (five independent seeds per arm, resampled independently, 10,000 resamples). A negative value favours the grid arm.

Opponent	Same examples: graph − grid	Same wall-clock: graph − grid
B-RND (legal-random)	−0.169 [−0.272, −0.062]	−0.161 [−0.278, −0.048]
B-HEU (heuristic)	−0.064 [−0.111, −0.017]	−0.059 [−0.087, −0.029]
B-MCTS (search, 6,400 simulations)	−0.009 [−0.055, +0.028]	−0.018 [−0.066, +0.025]

Seed by seed. Under the same-examples reading, every grid seed scores at least 0.949 against the legal-random opponent while graph seeds range from 0.688 to 0.981; against the heuristic, grid seeds range from 0.080 to 0.190 and graph seeds from 0.025 to 0.125; against the search opponent the two arms overlap (0.075–0.210 against 0.100–0.125). Four of the six contrasts exclude zero, all in the grid arm's favour; the two against the search opponent straddle zero. The picture persists at equal time. The three-seed analysis of 19 September 2026 had already rejected H1 under both readings; the extension to five seeds tightened four of the six intervals, absorbed the single graph-better seed pair observed against the heuristic (graph seed 4 at 0.125 against grid seed 4 at 0.105) and the best search-opponent score of any run (grid seed 5 at 0.210), and did not change the verdict. Against the search opponent, where both arms score around 0.1, five seeds per arm cannot distinguish the arms.

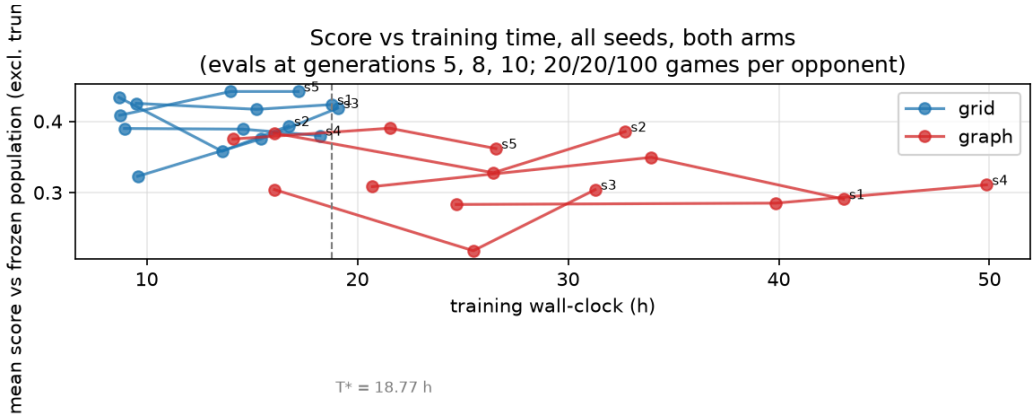


Figure 3: Mean score against the frozen population (average of the three opponents, truncations excluded) as a function of cumulative training wall-clock, for all ten main runs; evaluations at generations 5, 8 and 10 (20, 20 and 100 games per opponent). Blue: grid arm; red: graph arm; one line per seed. The dashed vertical line marks the equal-time cutoff of 18.77 hours.

The truncation channel. The graph arm truncated 57%, 20%, 55%, 44% and 23% of its games against the random opponent at the final checkpoint for seeds 1 to 5; the grid arm truncated 0%, 1%, 1%, 11% and 0%. No game of either arm against the heuristic or the search opponent reached the cap. Every graph seed truncates more than every grid seed (20–57% against 0–11%). The longest truncated game of the graph arm against the random opponent shows the mechanism: the arm wins material early, then shuffles pieces around a partially surrounded enemy queen until the cap, ahead throughout but never finding the forcing sequence that completes the surround. The pathology is visible only because truncation was defined as its own outcome before the first run, and it cannot rescue the hypothesis: under the strongest treatment in the graph arm's favour, which scores every truncated game as a win for the arm under test and so bounds what any larger cap could deliver, the graph–grid difference against the random opponent (computed from the per-game records of the three original seeds) is still −0.072 at equal examples and −0.058 at equal time, and the direction of the contrast is unchanged under every treatment. The graph arm's random-opponent score does rest on fewer decided games (between 43 and 80 per seed), which widens its interval.

Cost. Table 4 summarises what each arm costs on the study machine. The graph arm's single evaluation is 1.4× slower at its best provider, its training throughput is half that of the grid arm, and a full run costs it 2.0× the wall-clock (36.7 h against 18.0 h on average; the five grid seeds took 18.77, 16.73, 19.07, 18.23 and 17.19 hours and the five graph seeds 43.07, 32.71, 31.28, 49.86 and 26.57 hours). The population score (mean over the three opponents of the seed-level mean score) is 0.411 for the grid arm and 0.331 for the graph arm under the same-examples reading, 0.405 and 0.326 under the same-wall-clock reading: the same ordering and nearly the same gap, 0.081 and 0.079 in the grid arm's favour. The budget was read twice in case a representation learned more per example but less per hour; here the readings agree, and the equal-time reading enlarges a deficit that

the per-example reading already shows. The ten main runs consumed 273.5 h of training wall-clock (grid 90.0 h, graph 183.5 h), all on one laptop with no paid compute.

Table 4: Cost and score summary per arm. Training wall-clock covers self-play generation and training for the ten generations of a run, evaluation games excluded, averaged over the five seeds; inference and throughput are benchmark measurements of 10 September 2026 on the study machine (Apple M1 Pro, 10 cores, 16 GB). Population score = mean over the three frozen opponents of the seed-level mean score, truncations excluded.

Metric	**Grid arm**	**Graph arm**
Parameters	1.44 M	1.47 M (+1.5%)
Best-provider inference, batch 1	2.62 ms (neural accelerator)	3.67 ms (CPU)
Training wall-clock per run, mean of 5 seeds	18.0 h	36.7 h (2.0×)
Training throughput (batch 128, forward + backward)	274 pos/s	138 pos/s
Population score, same-examples reading	0.411	0.331
Population score, same-wall-clock reading (cutoff 18.77 h)	0.405	0.326

5.1 Ablations

Each ablation removes one component of the graph arm and nothing else, with three seeds per cell; differences are variant minus full graph arm, as seed-level means with seed-bootstrap intervals (10,000 resamples, independent seed sets).

With one shared message matrix in place of six typed ones (A1), training diverged to non-finite values in generation 0 for all three seeds, and every later generation self-played and trained on non-finite outputs. The symptom was statistical rather than numerical: the three seeds' final evaluations were identical down to the game count, which independently seeded trainings cannot produce. The cell's evaluation tables are therefore not strength measurements, and the result reads "training diverged (3/3 seeds)". The explanation we offer is a reasoned argument rather than a measured decomposition: a single shared matrix receives the summed gradient of six neighbour terms, roughly six times the per-matrix gradient scale of the typed variant, at an identical learning rate and momentum. No learning-rate sweep was run, because it would have made A1 a two-component difference.

Removing the global-pooling bias (A2) changed the score by −0.001 [−0.170, +0.165] against the random opponent, −0.007 [−0.043, +0.030] against the heuristic and −0.013 [−0.043, +0.013] against the search opponent. Every interval straddles zero, and the conversion pathology against the random opponent is present in both variants (truncation rates between 17% and 65% across seeds). At this scale the global-pooling bias is dispensable. The null is bounded by intervals of width ±0.17 against the random opponent, where a small effect could hide.

The stabilised naive-adjacency variant (A1′, untyped relations plus gradient clipping at 1.0, a two-component change) trained finitely in all three seeds and scored within the full graph arm's band, the full arm's five-seed means serving as reference: −0.066 [−0.254, +0.130], +0.032 [−0.011, +0.078] and +0.008 [−0.040, +0.063] against the three opponents, every interval straddling zero. Every number in this cell is confounded by the clip by construction and is never attributed to edge typing alone. Taken with A1, the most that

can be said is that at this scale the measurable contribution of direction-typed relations lies in optimization stability; no strength contribution beyond it is detectable.

5.2 The second study

Verdict. Under the warm-started regime the graph arm shows no seed-consistent advantage against any opponent under either reading (graph seeds outscore grid seeds in 2 to 3 of the 5 same-index pairs per opponent), and every interval on the graph–grid difference includes zero: Table 5 places the second study's contrasts beside the first study's. The largest upper bounds are +0.090 at equal examples, against the search opponent, where both arms score 0.13 with seeds spread from 0.045 to 0.260, and +0.055 at equal wall-clock. The frozen rule names no numeric threshold, so the bounds are stated: the rule rejects H1′ for any threshold above +0.090, and the data cannot exclude a graph advantage below about 0.09 against the search opponent at equal examples. H1′ is not supported.

Table 5: The graph–grid contrast of the two studies per frozen opponent and budget reading: difference of seed-level mean scores with the 95% bootstrap interval over seeds (five independent seeds per arm in each study, resampled independently, 10,000 resamples). First study: seeds 1 to 5, fresh initialisation each generation, cutoff 18.77 h. Second study: seeds 11 to 15, warm start and three-generation replay window, cutoff 17.63 h. The studies are not pooled.

Opponent	Reading	First study: graph − grid	Second study: graph − grid
B-RND	same examples	−0.169 [−0.272, −0.062]	−0.018 [−0.045, +0.008]
B-HEU	same examples	−0.064 [−0.111, −0.017]	−0.003 [−0.046, +0.039]
B-MCTS	same examples	−0.009 [−0.055, +0.028]	+0.003 [−0.082, +0.090]
B-RND	same wall-clock	−0.161 [−0.278, −0.048]	−0.052 [−0.140, +0.002]
B-HEU	same wall-clock	−0.059 [−0.087, −0.029]	+0.013 [−0.032, +0.055]
B-MCTS	same wall-clock	−0.018 [−0.066, +0.025]	−0.025 [−0.091, +0.034]

What moved. The contrast moved toward zero on every opponent under both readings, four contrasts that excluded zero now include it, and the sign changed in two cells inside intervals that include zero. The graph arm's mean against the random opponent rose from 0.812 to 0.971 at equal examples; its conversion pathology persists in three seeds of five (24–42% truncation against the random opponent at the final checkpoints, 28–58% at the cutoffs) and is absent in two (0–4%), the grid arm staying at 1–7%; with truncations scored 0.5 the random-opponent contrast is −0.101 [−0.187, −0.012] and −0.131 [−0.256, −0.015]. The strength reached against the strong opponents did not rise: the grid arm's mean against the heuristic is 0.077 [0.051, 0.104] against 0.129 [0.098, 0.165] in the first study, and against the search opponent 0.131 against 0.124. The graph arm cost 2.34 times the grid arm per run (41.59 h against 17.80 h on average; 296.9 h for the ten runs). The between-study movement is descriptive: different seed sets, two regime changes at once, no interval across studies.

6 Discussion

Reading the result. The deficit is opponent-shaped: largest against the legal-random opponent, clear against the heuristic, absent against the search opponent. A graph arm that was simply weaker everywhere would show a uniform deficit; a deficit concentrated where games are decided by finishing a surround that nothing resists points at local tactical conversion rather than positional judgement. The training metrics agree: both arms fit their self-play data throughout the ten generations, and the graph arm's value accuracy on non-truncated samples is comparable to the grid arm's while its policy top-1 trails. The gap is therefore not a plain optimization failure of the full graph arm; it separates two trained networks, one of which learns a usable evaluation and a weaker policy.

Plausible and demonstrated explanations. Geometry is plausible but not demonstrated: Hive's tactics are dominated by short-range surround geometry, the regime in which Keller et al. (2023) found convolutional networks stronger in Hex; a 3×3 convolution over the unfolded frame sees a cell's whole neighbourhood at no learning cost, whereas the message-passing network must compose the same patterns from six typed relations one hop at a time, and no ablation here isolates receptive-field composition. Capacity is controlled: the networks differ by +1.5%, in the graph arm's favour. The decoder is controlled: both arms score the identical legal set with identical masking, normalisation, targets and tie-breaking. Speed is demonstrated for the second reading only: the graph arm's generations cost roughly twice as much wall-clock, so at equal time it completes fewer than half of its generations, which widens the gap in the same-wall-clock reading and plays no role in the same-examples reading, where the deficit already exists. Optimization is partly demonstrated: the ablations show the graph arm's optimization to be fragile in one specific way (it depends on typed relations for stability), while the full arm fits its data in every seed; whether a different learning rate or normalisation would have favoured the graph arm was not tested, because neither arm received any tuning.

Relation to prior work. The result does not contradict the positive graph findings of Rigaux and Kashima (2024) in chess or the long-range advantage Keller et al. (2023) measured in Hex; it bounds where their optimism transfers. The chess result rests on a single training run per model, a loose capacity match and arm-specific decoders; the Hex result was obtained under a value-based learner with a game-specific graph construction, and found the convolutional arm sharper at local patterns, the regime that decides Hive games at small budgets. Our rejection is seed-consistent across five runs per arm under two pre-registered budget readings, with capacity matched and the decoder shared. Read together, the three studies suggest that the sign of the comparison depends on the game's tactical range and on the evidence standard rather than on the graph paradigm as such. AZ-Hive (de Goede et al., 2022) had shown that the encoding changes learning within the grid family; we extend the finding across families and find the grid side ahead.

Threats to validity. Internal: the engine is validated independently of learning, both encoders are pinned byte-exactly, and the arms share the decoder, the records, the losses, the budgets, the search and the frozen evaluation; the graph architecture is nevertheless one point in design space, and nothing is claimed beyond it. Tooling defects were found during the study (a match-runner path that dropped records, a training loop

that let non-finite values pass silently, and a results generator that averaged five runs' wall-clock over three); each was caught by mechanical verification, fixed and audited, and none touched a score, interval or claim. Measurement: scores are relative to three fixed opponents spanning floor-to-mid strength; all trained arms still lose heavily to the two strong opponents after ten generations, so differences against the search opponent cannot be resolved. Statistical: five seeds per arm bound the statistics; the rejection rests on seed-consistency across two opponents and two readings, on four of six intervals excluding zero and on the cap bounds, and not on a single interval. External: one variant (base Hive without expansions), one machine, one small budget (between 16.7 and 19.1 hours of training wall-clock per grid run and between 26.6 and 49.9 hours per graph run), early-regime self-play throughout. The zero-tuning policy is symmetric but may not be neutral: graph networks are commonly more sensitive to optimizer defaults than residual convolutional networks, and the first ablation showed that this family is fragile under these defaults; a tuned graph arm might narrow the gap, at the cost of breaking the symmetry of the comparison.

The re-initialisation property, and the second study. In the first study each generation's network was trained from a fresh seeded initialisation on the records of that generation alone, so each network saw the positions of 500 games, and the absolute strength reached after ten generations (near the ceiling against the random opponent, around 0.1 against the two strong opponents) must be read against that fact. For the comparison the property is neutral, being identical in both arms. The second study removed it, and the answer is in two parts. The ordering of the first study did not survive as a deficit: carried across generations and trained on a window, the graph representation plays the frozen population as well as the grid representation at equal examples, within five-seed intervals, and within 0.05 of it at equal time. It did not play better, and the strength reached against the strong opponents did not rise under the new regime, so the budget, and not only the regime, bounds what either representation shows here; both studies share the untuned defaults, and what the regime changed is the gap between the arms, not the level.

Two follow-ups. The first targets the conversion pathology, which the data isolate as a policy defect rather than a value defect: a search-time remedy applied identically to both arms under the same frozen evaluation, such as a deeper evaluation budget in positions the value head already judges won, or auxiliary training targets for forcing moves. The second concerns the stability finding: a controlled study of normalisation and gradient-scale choices for relation-shared graph layers would decouple trainability from representational content and say whether naive adjacency, properly stabilised, is sufficient for Hive. Either follow-up now has the warm-started regime as its baseline, and either should pre-register a numeric threshold for a meaningful advantage; the second study's wider intervals made the verdict depend on one.

7 Conclusion

Within the tested perimeter (base-game Hive, one capacity-matched relational message-passing network against one grid convolutional network, ten generations of small-budget self-play, five independent seeds per arm, a frozen three-opponent population

and 250 frozen openings), the answer to the main research question is no: the graph representation did not learn a better policy than the grid representation, neither at equal training examples nor at equal wall-clock on the same machine, and the pre-registered rejection rule fired exactly as it had been frozen. The deficit is largest where Hive is most tactical (−0.169 and −0.161 against the legal-random opponent under the two readings), clear against the heuristic (−0.064 and −0.059) and absent against the search opponent (−0.009 and −0.018). The graph arm pays roughly twice the wall-clock per run, and its characteristic failure is to win material without converting the win, which shows as 20–57% of its games against the random opponent ending at the move cap. Of its two distinctive components, the direction-typed relations are necessary for optimization itself, while the global-pooling bias is dispensable at this scale. Where short-range surround tactics decide games and budgets are small, the frame artifacts of a grid encoding turn out to be cheaper than framelessness. The second study, under the standard warm-started regime on five new seeds per arm, found no graph advantage under either reading and a cost ratio of 2.34: at this budget the graph representation did not learn a better policy than the grid representation in either training regime, and under the standard regime it learned an equivalent one, within five-seed intervals, at more than twice the cost.

Reproducibility and availability

The release set is one repository containing the Rust engine, the Python training and export code, the frozen configurations (comparison matrix, evaluation settings, opponent configurations with the hash-pinned heuristic weights, ablation specifications), the 250 frozen openings and the population manifest with its hashes, the result tables, and the per-run records: one file per checkpoint and opponent with one row per game, the per-generation wall-clock, the self-play manifests and shards, and the checkpoint of every generation. The third-party reference engines used in rules validation are MIT-licensed and are not shipped, since no study number derives from them. The first study was released on 9 October 2026 at tag v2.0-report of the public repository https://github.com/mohamedbechirkefi/hive-graph-selfplay; the second study's records accompany the release tagged v2.1-report.

A minimal fresh-environment scenario, run on 9 October 2026, performs from a clean clone plus the shipped records the smallest end-to-end check that touches every link of the chain: build the engine and the arena; replay one recorded game deterministically (the grid arm's seed-2 final checkpoint against the heuristic opponent on opening line 2, the arm playing Black, lost in 19 plies) and assert that its result row equals the shipped row; and regenerate both reading tables from the shipped records, byte-identical to the released ones. It passed. The scenario does not verify training; the full reproduction repeats the ten training runs (16.7–19.1 h per grid run and 26.6–49.9 h per graph run on the study machine), the cutoff evaluations and the ablations, the second study's ten warm-started runs (17.46–18.52 h and 36.27–52.85 h) with their cutoff evaluations, and regenerates every table and figure from the raw per-game records.

Declaration of assistance by an artificial-intelligence system. This study was executed under a human-as-principal-investigator methodology in which an AI research

assistant (Claude, Anthropic) implemented code, ran the computational campaigns, kept the experiment journals and drafted text, under a system of six decision gates reserving every scientific commitment to the human author: the freezing of protocol, opponents and openings, every expenditure of compute, anything made public and the deletion of any data. The human author chose the question, approved each gated decision in writing, verified the conclusions and owns every claim.

References

Agarwal, R., Schwarzer, M., Castro, P. S., Courville, A., & Bellemare, M. G. (2021). Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems* (NeurIPS 2021) (peer-reviewed; Outstanding Paper Award). arXiv:2108.13264 (v4, 5 January 2022). https://doi.org/10.48550/arXiv.2108.13264

Ben-Assayag, S., & El-Yaniv, R. (2021). Train on small, play the large: Scaling up board games with AlphaZero and GNN. arXiv preprint arXiv:2107.08387v1 (18 July 2021) (no peer-reviewed version found as of 26 September 2026). https://doi.org/10.48550/arXiv.2107.08387

de Goede, D., Kampert, D., & Varbanescu, A. L. (2022). The cost of reinforcement learning for game engines: The AZ-Hive case-study. In *Proceedings of the 13th ACM/SPEC International Conference on Performance Engineering* (ICPE 2022, Beijing), pp. 145–152. https://doi.org/10.1145/3489525.3511685

Hamilton, W. L., Ying, R., & Leskovec, J. (2017). Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems* (NIPS 2017) (peer-reviewed). arXiv:1706.02216 (v4, 10 September 2018). https://doi.org/10.48550/arXiv.1706.02216

Jones, A. L. (2021). Scaling scaling laws with board games. arXiv preprint arXiv:2104.03113v2 (15 April 2021) (not peer-reviewed). https://doi.org/10.48550/arXiv.2104.03113

Kampert, D., Varbanescu, A.-L., Müller-Brockhausen, M., & Plaat, A. (2021). Mimicking the human approach in the game of Hive. In *2021 IEEE Symposium Series on Computational Intelligence* (SSCI 2021, Orlando). IEEE Xplore document 9659999. https://ieeexplore.ieee.org/document/9659999/. Preprint circulated as "Better AI for Hive: Mimicking human game-play strategies".

Keller, Y., Blüml, J., Sudhakaran, G., & Kersting, K. (2023). From images to connections: Can DQN with GNNs learn the strategic game of Hex? arXiv preprint arXiv:2311.13414 (22 November 2023) (not peer-reviewed; OpenReview submission dYaeDrazj5). https://arxiv.org/abs/2311.13414

Rigaux, T., & Kashima, H. (2024). Enhancing chess reinforcement learning with graph representation. In *Advances in Neural Information Processing Systems 37* (NeurIPS 2024, main conference track). https://doi.org/10.52202/079017-0006. Preprint: arXiv:2410.23753v1 (31 October 2024), https://doi.org/10.48550/arXiv.2410.23753

Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K., & Hassabis, D. (2018). A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419), 1140–1144 (peer-reviewed). https://doi.org/10.1126/science.aar6404.

Preprint (2017): Mastering chess and shogi by self-play with a general reinforcement learning algorithm, arXiv:1712.01815v1 (5 December 2017), https://doi.org/10.48550/arXiv.1712.01815

Vinyals, O., Fortunato, M., & Jaitly, N. (2015). Pointer networks. In *Advances in Neural Information Processing Systems 28* (NIPS 2015) (peer-reviewed). arXiv:1506.03134 (v2, 2 January 2017). https://arxiv.org/abs/1506.03134

Wu, D. J. (2020). Accelerating self-play learning in Go. arXiv preprint arXiv:1902.10565v5 (9 November 2020); presented at the AAAI-20 Workshop on Reinforcement Learning in Games (not a full peer-reviewed proceedings paper). https://doi.org/10.48550/arXiv.1902.10565

Web resources.

edre (GitHub user). *nokamute*: Hive engine in Rust, with a Universal Hive Protocol conformance tester and a built-in match runner [source-code repository]. GitHub. https://github.com/edre/nokamute. Version 1.0.3 used as a reference engine. Consulted July 2026 (engine design) and 9 September 2026 (validation campaign).

jonthysell (GitHub user). *Mzinga*: reference Hive engine and project wiki, including the Universal Hive Protocol specification and the perft tables [source-code repository and wiki]. GitHub. https://github.com/jonthysell/Mzinga; perft tables: https://github.com/jonthysell/Mzinga/wiki/Perft. Release MzingaEngine v0.16.0 used as a reference engine. Consulted July 2026 (engine design) and 9 September 2026 (validation campaign).

World Hive Tournaments. *Rules of Hive: Rules FAQ* [web page]. https://www.worldhivetournaments.com/rules-of-hive/. Consulted 9 September 2026.

Yianni, J. (2010). *Hive rules* [publisher's rules sheet]. Gen42 Games. https://www.gen42.com/wp-content/uploads/Hive-rules.pdf. Consulted 9 September 2026.